arXiv:2410.02707v4 [cs.CL] 18 May 2025

LLMs KNOW MORE THAN THEY SHOW: ON THE INTRINSIC REPRESENTATION OF LLM HALLUCINATIONS

Hadas Orgad[1]* **Michael Toker**[1] **Zorik Gekhman**[1] **Roi Reichart**[1]
Idan Szpektor[2] **Hadas Kotek**[3] **Yonatan Belinkov**[1]
[1]Technion [2]Google Research [3]Apple

ABSTRACT

Large language models (LLMs) often produce errors, including factual inaccuracies, biases, and reasoning failures, collectively referred to as "hallucinations". Recent studies have demonstrated that LLMs' internal states encode information regarding the truthfulness of their outputs, and that this information can be utilized to detect errors. In this work, we show that the internal representations of LLMs encode much more information about truthfulness than previously recognized. We first discover that the truthfulness information is concentrated in specific tokens, and leveraging this property significantly enhances error detection performance. Yet, we show that such error detectors fail to generalize across datasets, implying that—contrary to prior claims—truthfulness encoding is not universal but rather multifaceted. Next, we show that internal representations can also be used for predicting the types of errors the model is likely to make, facilitating the development of tailored mitigation strategies. Lastly, we reveal a discrepancy between LLMs' internal encoding and external behavior: they may encode the correct answer, yet consistently generate an incorrect one. Taken together, these insights deepen our understanding of LLM errors from the model's internal perspective, which can guide future research on enhancing error analysis and mitigation.[1]

1 INTRODUCTION

The ever-growing popularity of large language models (LLM) across many domains has brought a significant limitation to center stage: their tendency to "hallucinate" – which is often used to describe the generation of inaccurate information. But what are hallucinations, and what causes them? A considerable body of research has sought to define, taxonomize, and understand hallucinations through extrinsic, behavioral analysis, primarily examining how users perceive such errors (Bang et al., 2023; Ji et al., 2023; Huang et al., 2023a; Rawte et al., 2023). However, this approach does not adequately address how these errors are encoded within the LLMs. Alternatively, another line of work has explored the internal representations of LLMs, suggesting that LLMs encode signals of truthfulness (Kadavath et al., 2022; Li et al., 2024; Chen et al., 2024, *inter alia*). However, these analyses were typically restricted to detecting errors—determining whether a generated output contains inaccuracies—without delving deeper into how such signals are represented and could be leveraged to understand or mitigate hallucinations.

In this work, we reveal that the internal representations of LLMs encode much more information about truthfulness than previously recognized. Through a series of experiments, we train classifiers on these internal representations to predict various features related to the truthfulness of generated outputs. Our findings reveal the patterns and types of information encoded in model representations, linking this intrinsic data to extrinsic LLM behavior. This enhances our ability to detect errors (while understanding the limitations of error detection), and may guide the development of more nuanced strategies based on error types and mitigation methods that make use of the model's internal knowledge. Our experiments are designed to be general, covering a broad array of LLM limitations. While the term "hallucinations" is widely used, it lacks a universally accepted definition (Venkit et al., 2024). Our framework adopts a broad interpretation, considering hallucinations to encompass

*Corresponding author; Work partially done during internship at Apple.
[1]Our code is available in `https://github.com/technion-cs-nlp/LLMsKnow`.

all errors produced by an LLM, including factual inaccuracies, biases, common-sense reasoning failures, and other real-world errors. This approach enables us to draw general conclusions about model errors from a broad perspective.

Our first step is identifying *where* truthfulness signals are encoded in LLMs. Previous studies have suggested methods for detecting errors in LLM outputs using intermediate representations, logits, or probabilities, implying that LLMs may encode signals of truthfulness (Kadavath et al., 2022; Li et al., 2024; Chen et al., 2024). Focusing on long-form generations, which reflect real-world usage of LLMs, our analysis uncovers a key oversight: the choice of token used to extract these signals (Section 3). We find that **truthfulness information is concentrated in the exact answer tokens** – e.g., "Hartford" in "The capital of Connecticut is Hartford, an iconic city...". Recognizing this nuance **significantly improves error detection** strategies across the board, revealing that truthfulness encoding is stronger than previously observed.

From this point forward, we concentrate on our most effective strategy: a classifier trained on intermediate LLM representations within the exact answer tokens, referred to as 'probing classifiers' (Belinkov, 2021). This approach helps us explore what these representations reveal about LLMs. Our demonstration that a trained probing classifier can predict errors suggests that LLMs encode information related to their own truthfulness. However, we find that probing classifiers do not generalize across different tasks (Section 4). Generalization occurs only within tasks requiring similar skills (e.g., factual retrieval), indicating the **truthfulness information is "skill-specific" and varies across different tasks**. For tasks involving different skills, e.g., sentiment analysis, these classifiers are no better–or worse–than logit-based uncertainty predictors, challenging the idea of a "universal truthfulness" encoding proposed in previous work (Marks & Tegmark, 2023; Slobodkin et al., 2023). Instead, our results indicate that LLMs encode multiple, distinct notions of truth. Thus, deploying trainable error detectors in practical applications should be undertaken with caution.

We next find evidence that **LLMs encode not only error detection signals but also more nuanced information about error types**. Delving deeper into errors within a single task, we taxonomize its errors based on responses across repeated samples (Section 5). For example, the same error being consistently generated is different from an error that is generated occasionally among many other distinct errors. Using a different set of probing classifiers, we find that error types are predictable from the LLM representations, drawing a connection between the models's internal representations and its external behavior. This classification offers a more nuanced understanding of errors, enabling developers to predict error patterns and implement more targeted mitigation strategies.

Finally, we find that the truthfulness signals encoded in LLMs can also differentiate between correct and incorrect answers for the same question (Section 6). Results highlight a significant misalignment between LLM's internal representations and its external behavior in some cases. **The model's internal encoding may identify the correct answer–yet it frequently generates an incorrect response**. This discrepancy reveals that the LLM's external behavior may misrepresent its abilities, potentially pointing to new strategies for reducing errors by utilizing its existing strengths. Overall, our model-centric framework provides a deeper understanding of LLM errors, suggesting potential directions for improvements in error analysis and mitigation.

2 BACKGROUND

Defining and characterizing LLM errors. The term "hallucinations" is widely used across various subfields such as conversational AI (Liu et al., 2022), abstractive summarization (Zhang et al., 2019), and machine translation (Wang & Sennrich, 2020), each interpreting the term differently. Yet, no consensus exists on defining hallucinations: Venkit et al. (2024) identified 31 distinct frameworks for conceptualizing hallucinations, revealing the diversity of perspectives. Research efforts aim to define and taxonomize hallucinations, distinguishing them from other error types (Liu et al., 2022; Ji et al., 2023; Huang et al., 2023a; Rawte et al., 2023). On the other hand, recent scholarly conversations introduce terms like "confabulations" (Millidge, 2023) and "fabrications" (McGowan et al., 2023), attributing a possible "intention" to LLMs, although the notions of LLM "intention" and other human-like traits are still debated (Salles et al., 2020; Serapio-García et al., 2023; Harnad, 2024). These categorizations, however, adopt a *human-centric* view by focusing on the subjective interpretations of LLM hallucinations, which does not necessarily reflect how these errors are encoded within the models themselves. This gap limits our ability to address the root causes of

hallucinations, or to reason about their nature. For example, it is unclear whether conclusions about hallucinations defined in one framework can be applied to another framework. Liang et al. (2024) defined hallucinations as inconsistencies with the training data. While this approach engage with the possible root causes of hallucinations, our study focuses on insights from the model itself, without requiring training data access. Instead, we adopt a broad interpretation of hallucinations. Here, we define hallucinations as any type of error generated by an LLM, including factual inaccuracies, biases, failures in common-sense reasoning, and others.

Another line of research suggests that LLMs either encode information about their own errors (Kadavath et al., 2022; Azaria & Mitchell, 2023) or exhibit discrepancies between their outputs and internal representations (Liu et al., 2023; Gottesman & Geva, 2024), indicating the presence of underlying mechanisms not reflected in their final outputs. Moreover, Yona et al. (2024) found that current LLMs fail to effectively convey their uncertainty through their generated outputs. Hence, we propose shifting the focus from human-centric interpretations of hallucinations to a *model-centric* perspective, examining the model's intermediate activations.

Error detection in LLMs. Error detection is a longstanding task in NLP, crucial for maintaining high standards in various practical applications and for constructing more reliable systems that ensure user trust (Bommasani et al., 2021). Over the years, many studies have proposed task-specific solutions (see Section A.1). However, the recent shift towards general-purpose LLMs necessitates a holistic approach capable of addressing any error type, rather than focusing on specific ones, making it suitable for the diverse errors generated by these models.

A line of work has addressed this challenge by leveraging external knowledge sources (Lewis et al., 2020; Gao et al., 2023) or an external LLM judge (Lin et al., 2021; Rawte et al., 2023) to identify erroneous outputs. On the other hand, our work focuses on detection methods that rely solely on the computations of the LLM—specifically, output logits, probabilities after softmax, and hidden states.

Error detection in LLMs is also closely linked to uncertainty estimation, where low certainty signals potential inaccuracies and possible errors. Popular methods to derive calibrated confidence include inspecting the model logit output values (Varshney et al., 2023; Taubenfeld et al., 2025), agreement across multiple sampled answers (Kuhn et al., 2023; Manakul et al., 2023; Tian et al., 2023a), verbalized probability (Tian et al., 2023b), and direct prompting (Kadavath et al., 2022).

Another line of work trains probing classifiers to discover and utilize truthfulness features. This approach has shown some success by probing the final token of an answer–either generated (Kadavath et al., 2022; Snyder et al., 2023; Yuksekgonul et al., 2023; Zou et al., 2023; Yin et al., 2024; Chen et al., 2024; Simhi et al., 2024; Gekhman et al., 2025) or not (Li et al., 2024; Marks & Tegmark, 2023; Burns et al., 2022; Azaria & Mitchell, 2023; Rateike et al., 2023). Others probe the final token of the prompt before the response is generated (Slobodkin et al., 2023; Snyder et al., 2023; Simhi et al., 2024; Gottesman & Geva, 2024). Many previous studies simplify the analysis by generating answers in a few-shot setting or limiting generation to a single token. In contrast, we simulate real-world usage of LLMs by allowing unrestricted answer generation. By probing exact answer tokens, we achieve significant improvements in error detection.

3 Better Error Detection

This section presents our experiments on detecting LLM errors through their own computations, focusing on token selection's impact and introducing a method that outperforms other approaches.

3.1 Task Definition

Given an LLM M, an input prompt p and the LLM-generated response $\hat{y}$, the task is to predict whether $\hat{y}$ is correct or wrong. We assume that there is access to the LLM's internal states (i.e., white-box setting), but no access to any external resources (e.g., search engine or additional LLMs).

We use a dataset $D = \{(q_i, y_i)\}_{i=1}^{N}$, consisting of N question-label pairs, where $\{q_i\}_{i=1}^{N}$ represents a series of questions (e.g., *"What is the capital of Connecticut?"*) and $\{y_i\}_{i=1}^{N}$ the corresponding ground-truth answers (*"Hartford"*). For each question q_i, we prompt the model M to generate a response y_i, resulting in the set of predicted answers $\{\hat{y}_i\}_{i=1}^{N}$ (*"The capital of Connecticut is Hart-*

ford..."). Next, to build our error-detection dataset, we evaluate the correctness of each generated response $\hat{y}_i$ by comparing it to the ground-truth label y_i. This comparison yields a correctness label $z_i \in \{0, 1\}$ (1 correct, 0 wrong). The comparison can be done either via automatic heuristics or with the assistance of an instruct-LLM.[2] Our error detection dataset is: $\{(q_i, \hat{y}_i, z_i)\}_{i=1}^{N}$. Note that this dataset is defined based on the analyzed LLM and its generated answers. Any instances where the LLM refuses to answer are excluded, as these can easily be classified as incorrect.

3.2 Experimental Setup

Datasets and models. We perform all experiments on four LLMs: Mistral-7b (Jiang et al., 2023), Mistral-7b-instruct-v0.2 (denoted Mistral-7b-instruct), Llama3-8b (Touvron et al., 2023), and Llama3-8b-instruct. We consider 10 different datasets spanning various domains and tasks: TriviaQA (Joshi et al., 2017), HotpotQA with/without context (Yang et al., 2018), Natural Questions (Kwiatkowski et al., 2019), Winobias (Zhao et al., 2018), Winogrande (Sakaguchi et al., 2021), MNLI (Williams et al., 2018), Math (Sun et al., 2024), IMDB review sentiment analysis (Maas et al., 2011), and a dataset of movie roles (movies) that we curate. We allow unrestricted response generation to mimic real-world LLM usage, with answers decoded greedily. For more details on the datasets and the prompts used to generate answers, refer to Appendix A.3.

Performance metric. We measure the area under the ROC curve to evaluate error detectors, providing a single metric that reflects their ability to distinguish between positive and negative cases across many thresholds, balancing sensitivity (true positive rate) and specificity (false positive rate).

Error detection methods. We compare methods from both uncertainty and hallucinations literature.

- **Aggregated probabilities / logits:** Previous studies (Guerreiro et al., 2023; Kadavath et al., 2022; Varshney et al., 2023; Huang et al., 2023b) aggregate output token probabilities or logits to score LLM confidence for error detection. We implement several methods from the literature, calculating the minimum, maximum, or mean of these values. The main paper reports results for the most common approach, **Logits-mean**, and the best-performing one, **Logits-min**, with additional baselines in Appendix B.
- **P(True):** Kadavath et al. (2022) showed that LLMs are relatively calibrated when asked to evaluate the correctness of their generation via prompting. We implement this evaluation using the same prompt.
- **Probing:** Probing classifiers involve training a small classifier on a model's intermediate activations to predict features of processed text (Belinkov, 2021). Recent studies show their effectiveness for error detection in generated text (Kadavath et al., 2022, *inter alia*). An intermediate activation is a vector $h_{l,t}$ from a specific LLM layer l and (either read or generated) token t. Thus, each LLM generation produces multiple such activations. Following prior work, we use a linear probing classifier for error detection (Li et al., 2024, inter alia) on static tokens: the last generated token ($h_{l,-1}$), the one before it ($h_{l,-2}$), and the final prompt token ($h_{l,k}$). The layer l is selected per token based on validation set performance.

For further details on the implementation of each method, refer to Appendix A.4.

Exact Answer Tokens. Existing methods often overlook a critical nuance: the token selection for error detection, typically focusing on the last generated token or taking a mean. However, since LLMs typically generate long-form responses, this practice may miss crucial details (Brunner et al., 2020). Other approaches use the last token of the prompt (Slobodkin et al., 2023, *inter alia*), but this is inherently inaccurate due to LLMs' unidirectional nature, failing to account for the generated response and missing cases where different sampled answers from the same model vary in correctness. We investigate a previously unexamined token location: the *exact answer tokens*, which represent the most meaningful parts of the generated response. We define exact answer tokens as those whose modification alters the answer's correctness, disregarding subsequent generated content.[3] Figure 1 illustrates the different token locations. In the following experiments, we implement

[2]For most datasets, we use heuristics to predict correctness, except for one case. See Appendix A.2.

[3]In practice, we do not use this definition for extracting the exact answer, but rather an instruct model in a few-shot setting. Still, the definition is useful to manually verify that automatic extractions work as expected.